Distributed Rate Limiter Engine

Patterns: Strategy Pattern, Factory Pattern, Singleton Pattern, Token Bucket | Domain: Infrastructure / Edge Traffic Control

Core Requirements & Features	Core Domain Classes & Interfaces
• **Algorithm Variety:** Token Bucket, Leaky Bucket, Sliding Window Counter algorithms. • **Configurable Limits:** Enforce per-user, per-IP, or per-API endpoint limits (e.g. 100 req/min). • **Thread Safety & Low Latency:** Sub-millisecond decision latency under high concurrency.	• **RateLimiterService:** Main entry point evaluating allowRequest(clientId). • **IRateLimiter:** Strategy interface for rate limiting algorithms. • **TokenBucketLimiter:** Token bucket implementation with refilling logic. • **RateLimiterFactory:** Factory producing limiter strategy instances.

Critical Execution Workflow & Method Trace

1. Client request arrives at API Gateway for user_101.
2. RateLimiterService delegates to TokenBucketLimiter strategy.
3. TokenBucketLimiter fetches user's TokenBucket state (tokens, lastRefillTimestamp).
4. Calculates refilled tokens = (currentTime - lastRefillTimestamp) * refillRate.
5. Updates tokens = min(capacity, currentTokens + refilledTokens).
6. If tokens >= 1: Decrements tokens -= 1, updates lastRefillTimestamp, returns ALLOWED (true).
7. If tokens < 1: Returns DENIED (false); Gateway responds with HTTP 429 Too Many Requests.

Concurrency & Thread Safety Mechanics	Design Trade-offs & Interview FAQs
• **Atomic Token Operations:** AtomicLong / AtomicReference for thread-safe state mutations. • **Redis + Lua Script:** Atomic execution in distributed setups (prevents read-modify-write race condition). • **ConcurrentMap Storage:** ConcurrentHashMap for multi-tenant isolation.	• **Token Bucket vs Sliding Window:** Token Bucket allows traffic bursts; Sliding Window Counter provides smoother distribution. • **Memory Overhead:** Sliding Window Log stores every timestamp (high RAM); Sliding Window Counter uses static counter (low RAM).